

목차

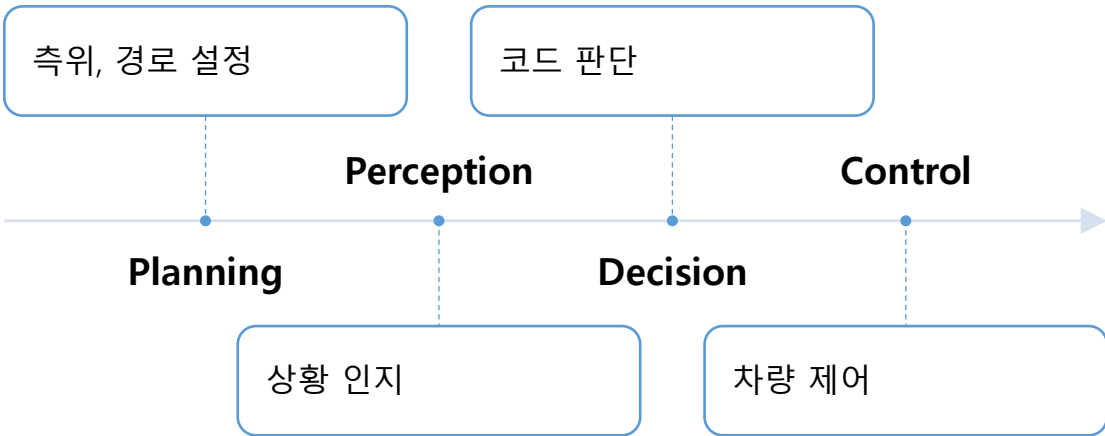
1. 센서 및 차량 사양
 - a) Hardware
 - b) Software

2. 평시 주행 - Localization, Path planning
 - a) 차량위치 측정 (Localization – 센서 퓨전, 칼만 필터)
 - b) 주행 알고리즘 (Stanley, Pure Pursuit)
 - c) Global Path Planning (HD Map, origin offset)

3. 상황 인식, Vision
 - a) Object detection (Camera)
 - b) Lane detection (Camera)
 - c) Obstacle avoidance (LiDAR)
 - d) Parking, Delivery (Camera + LiDAR)

4. Work Flow Chart

- 학교를 대표하여 대회에 처음 참여하게 되었습니다. 팀원들과 서로 도와고, 호기심과 열정을 잃지 않으며, 창의적인 도전을 시도하고자 자율주행 대회에 참여하게 되었습니다.
- 대회 참여를 통해 교내에서 자율주행 대회 참여를 독려하고, 모빌리티 자율주행 분야의 역량을 향상시킬 수 있는 기회가 될 수 있으면 좋겠습니다.



1. 활용 Hw/Sw

1. 센서 및 차량 사양

a) Hardware

I. 센서

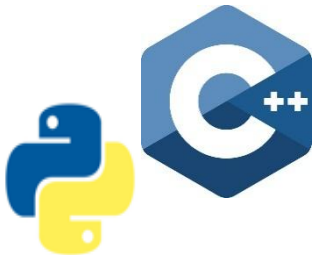
구분	품명	용도	예정 수량
Usb_camera	Logitech C920S HD Pro	이미지 탐지	5대
IMU	VECTORNAV VN-100	가속도 센서	1대
GPS	UBLOX EVK-F9P-01	위치 측정	1대
3D LiDAR	Velodyne-VLP16	주변 사물 인식	1대
Laptop	LG 울트라기어	소프트웨어 구동	1대



b) Software



Package, Node 등
센서 활용 및
차량 제어에 활용



프로그래밍 언어.
Package 구성
Script로 주로 활용

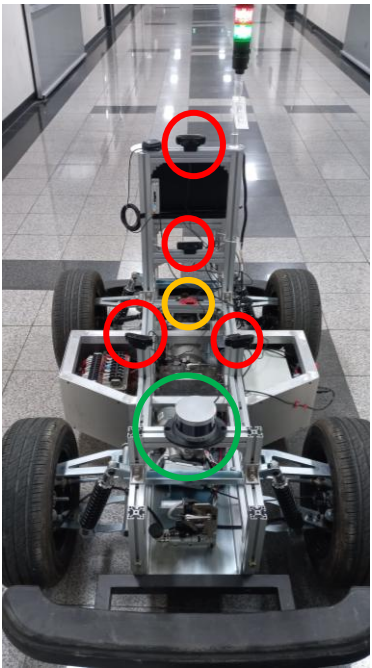


OpenCV
카메라에 Vision 적용을
위한 이미지 분석,
머신러닝 소프트웨어

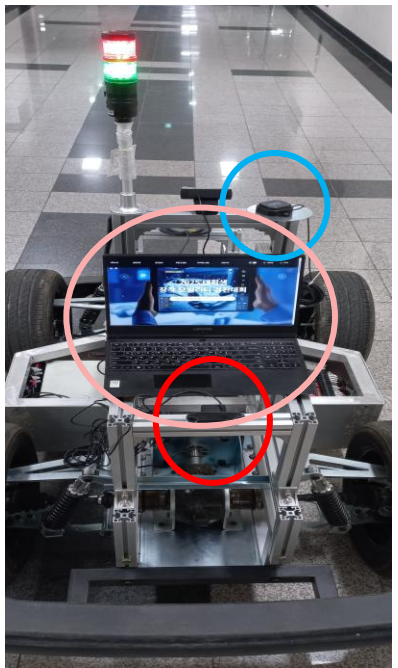


II. 활용 차량: ERP-42

- WEGO 사의 자율주행 실습 전기 차량
- 조향 각도, 속도 등을 조절함으로써 차량의 도로 자율주행 구현 가능
- 리모트 컨트롤러를 활용해 직접 조작 및 E-Stop 가능
- 전방 Alignment 조정 등 정비 가능



Front View



Rear View

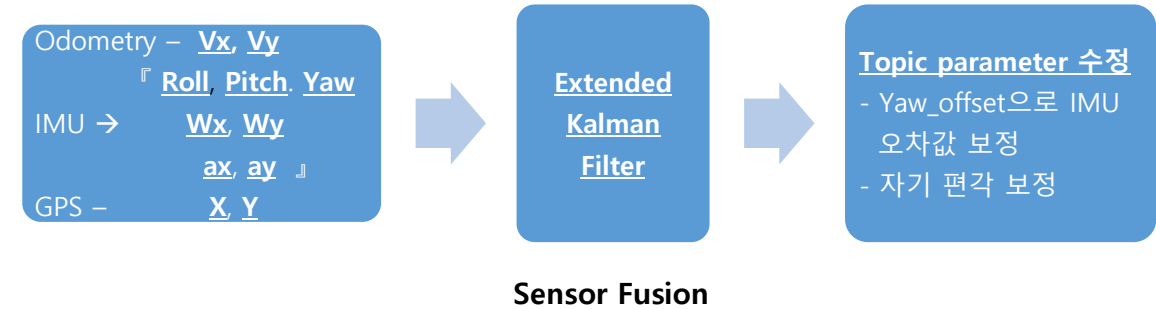
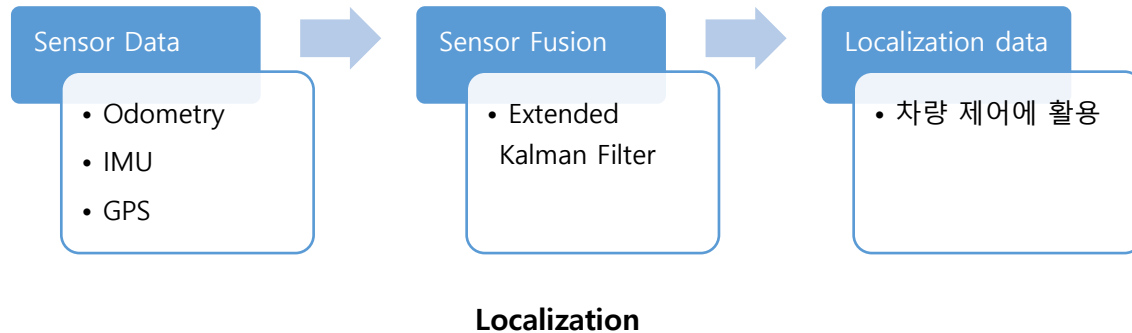
III. 프레임 및 센서 부착

- 알루미늄 프로파일 및 직각 브라켓으로 안정된 구조 확보
- 각 센서의 탐지 범위 및 용도를 고려한 구조 설계
- 경진대회 플랫폼 허용 사양 적용

2. 평시 주행 - Localization

● 위치 추정 (Localization)

- 각 센서의 측정 데이터를 보정. 이후 Sensor Fusion을 적용
 - 정확한 위치, 방향, 속도 데이터로 정밀한 차량 제어를 수행
1. 각 센서 데이터 생성
 - Odometry : 바퀴 Encoder 회전 당 이동거리, 속도 측정
 - IMU : 설치 위치에 대한 오차 측정, 센서 보정
 - GPS : 위성항법 시스템(GNSS) 적용으로 센서 보정
 2. Sensor Fusion 적용
 3. 필터링이 적용된 /global, /local 토픽 생성
 - /global : 경로주행 및 위치 추정에 활용. UTM좌표계 상의 위치 추정
 - /local : 센서음영지대(터널) 등에서 차량 위치 추정



● Sensor Fusion

- 센서 데이터를 조합해 필터를 적용, 신뢰할 수 있는 결과 데이터 생성
 - 비선형 시스템이므로, 확장 칼만 필터를 적용.
 - 객체의 다음 상태를 예측, 실제 모델과 예측 데이터 간의 오류를 비교 및 보정하여 결과데이터 생성
1. 센서 데이터 선정. 행렬조합 생성
 2. Extended Kalman Filter 적용, Covariance(공분산) 조절
 3. 결과 Topic Parameter 수정
 - Yaw_offset : 방향보정 - IMU 보정 소프트웨어로 오차 확인, 보정
 - Magnetic_declination_radaus: 지자계 구역 설정(구역 수치 적용)

2. 평시 주행 - 주행경로 설정

● 경로 계획 (Global Path-planning)

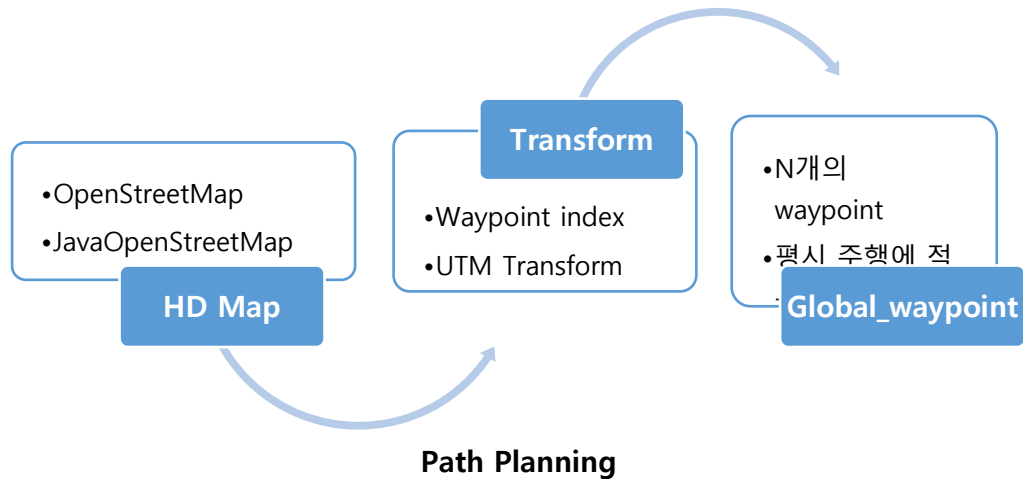
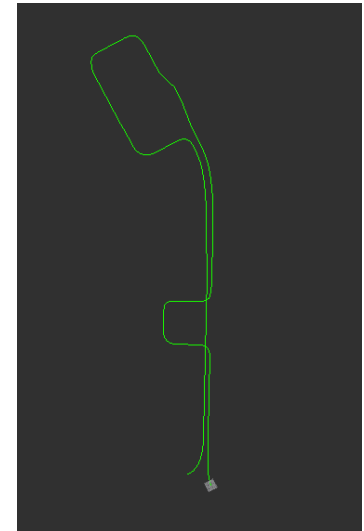
- 고정밀 위성 지도(HighDefinition Map)을 활용하여 차량의 주행 경로를 설정
- 위치 추정 데이터(Localization)를 기반으로 차량의 이동을 제어

- JOSM(JavaOpenStreetMap)을 활용해 도로경로 생성
- 경로의 ID값을 추출하여, 위·경도값 지정 및 순번 부여(.xls 엑셀파일)
- 지정된 위치값을 UTM좌표계로 변환
- 변환한 waypoint들을 Topic data로 Publish

K-City HD Map



RVIZ waypoint



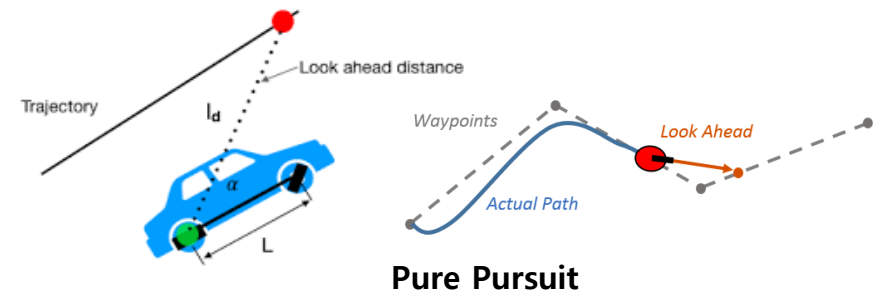
● Planning Practice

- 설계한 Path-Planning으로 실제 도로 주행이 가능한 지 연습
- Global 좌표계에서 정확한 경로주행을 위해, GPS (Convariance ≥ 0.02) 위도경도 측정으로 path planning 연습
- 엑셀파일에 위치정보 기입, 순서 부여로 경로 설정
- /waypoint의 publish에 Service통신 대입으로, 경로 시작위치와 차량 위치정보를 갱신 및 실습 수행

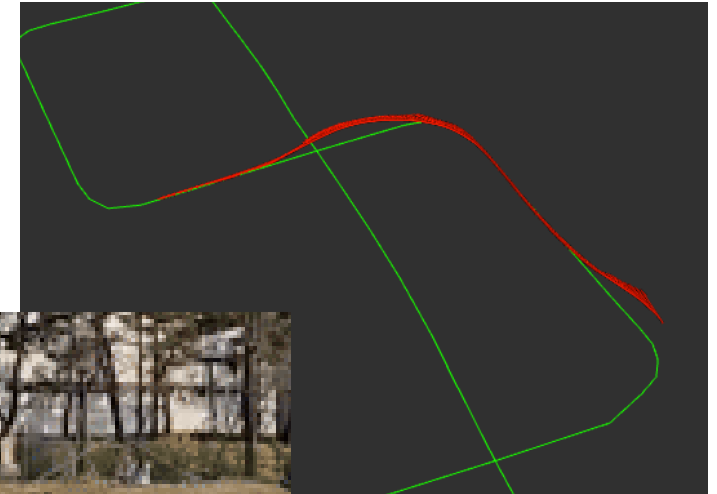
2. 평시 주행 – 차량 주행 제어

● 차량 주행 제어

- 종방향 제어
 - 직진 및 좌우회전 구간, 특정 부여 상황에 따라 속도값 제어
 - PID 제어를 적용해 차량의 속도 조절
- 횡방향 제어
 1. Pure Pursuit, Stanley method: 목표 waypoint와 차량 현재위치 간의 각도 차이를 계산, 차량 최대 조향 각도를 고려하여 차량의 조향 각도를 제어.
 2. 차량 전방에 LookAheadDistance 적용으로, 원활하게 경로를 추적하도록 설계.
 3. 계산한 값으로 차량의 Steering 각도를 조절.
 - 차량 위치 데이터의 Quaternion 데이터를 Euler 각으로 변환하여, Heading Error 계산
 - P제어 가중치 부여로 실제 조향 각도를 조절
 - 저역필터 적용으로, 차량의 급격한 조향 진동 최소화



Path tracking



Green – Path
Red – Vehicle Position

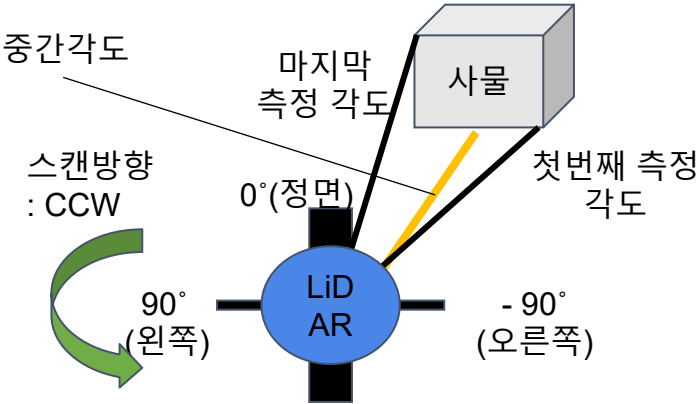


- ❖ 각 미션을 수행하여 완주하는 데에 필요한 부분
 - ❖ 미션 수행 시에는, local path를 사용하므로 일반 주행 상황과 별개의 Path를 사용
 - ❖ 상황 이후 정상 주행을 위해, 차량 현재위치와 가장 가까운 waypoint를 계산
 - ❖ 적절한 인근의 waypoint를 탐색하여 다음 경로로 원활히 주행할 수 있도록 제어

3. 상황 인식 – LiDAR

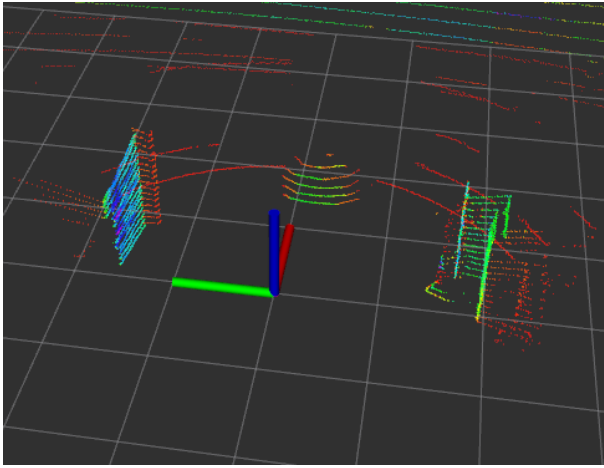
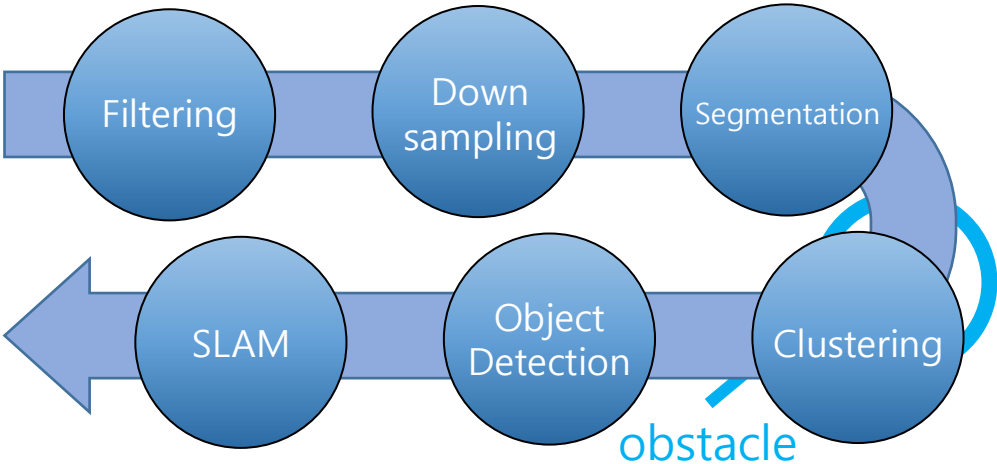
● Obstacle avoidance (LiDAR)

LiDAR 데이터 처리 방식	- 스캔 방식 : 반시계 방향, $-180^{\circ} \sim +180^{\circ}$ - 측정 데이터 : intensity (반사강도), range (거리) - 처리 방식 : 장애물 index간의 간격을 통해 빈 공간을 탐지하고 Local Planning으로 회피 경로 설정
ROI 설정	- ROI 설정: 관심 영역을 설정하여 필요 영역만 인식 - 거리 : 0 ~ 5m 이내 - 각도 : $-60^{\circ} \sim +60^{\circ}$
중심각 계산 및 주행 방향 결정	- 중심각 계산: 최근 두 장애물 감지 각도의 차이가 5°이상이면, 중간각도를 저장 - 주행 방향 결정 : 좌/우 빈 공간을 비교해 더 넓은 방향으로 주행하며, 만약, 두 장애물 사이의 공간이 더 넓으면 장애물 사이의 중간각도를 주행각도로 설정



Obstacle avoidance

● PointCloud 처리



Point Cloud data

3. 상황 인식 – Camera Vision

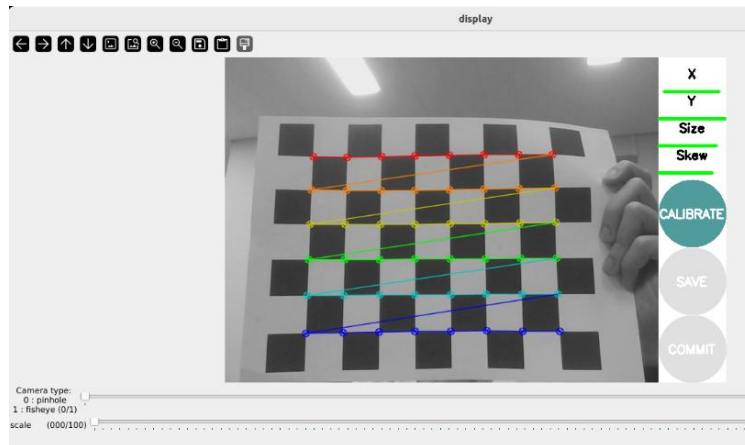
● 영상 기반 인식과 정밀화 기술 (Camera)

Camera Calibration

- 화면 정규화와 **격자 이미지**를 활용한 캘리브레이션을 통해 카메라 왜곡 보정
- 다양한 각도나 거리에서 표지판을 **왜곡 없이 안정적으로 인식**

Multiplex Camera

- 각 카메라에 **고유 번호와 포트를 할당**하여 여러 대의 카메라를 동시에 운용할 수 있도록 구성
- 다중 카메라 구성을 통해 **넓은 시야 확보와 사각지대 제거** 등 자율주행 인지 성능 강화



Camera Calibration

● 딥러닝 기반 교통 신호 및 상황 인지 시스템 (Camera, YOLOv8)

데이터셋 학습

- 교통 신호를 명확히 라벨링한 이미지로 학습

이미지 입력

- 카메라에서 들어온 영상을 YOLOv8 모델에 입력

피쳐 추출

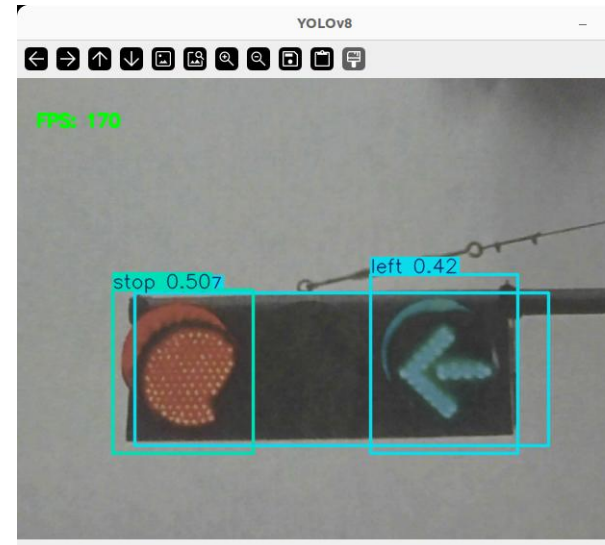
- 모델의 backbone이 이미지의 특징 추출

객체 예측

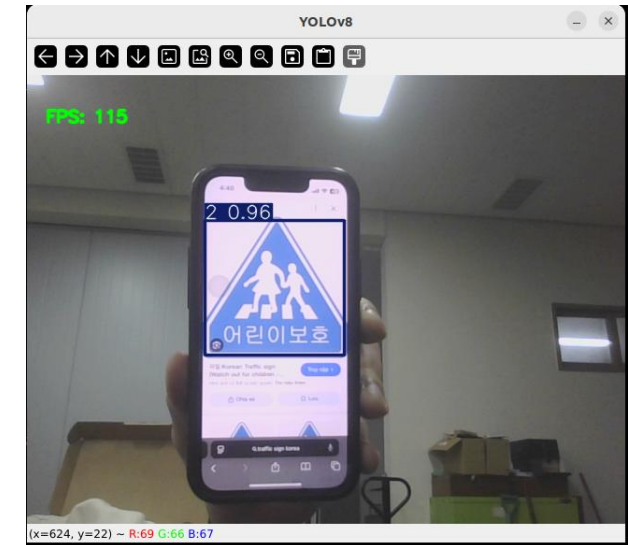
- Head 네트워크가 객체 위치 및 클래스 출력

차량 제어

- 인식 상황별로 적절한 차량 제어 명령 출력

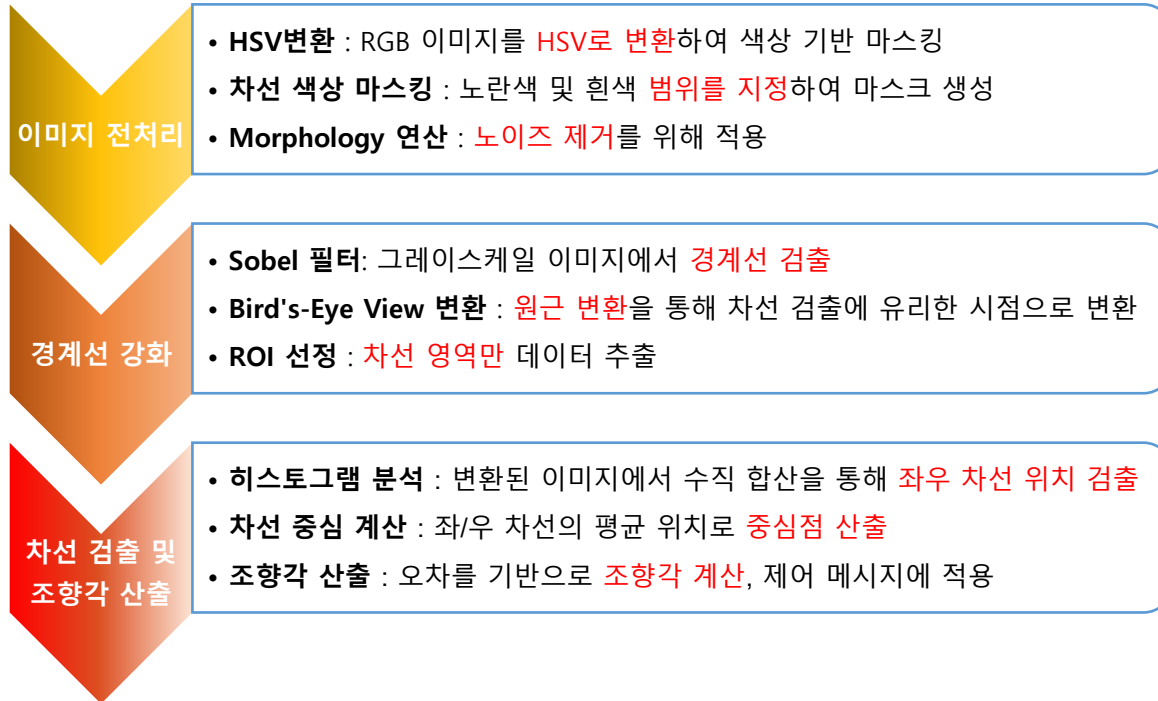


YOLOv8을 적용한 Traffic Detection



3. 상황 인식 – Lane Detection

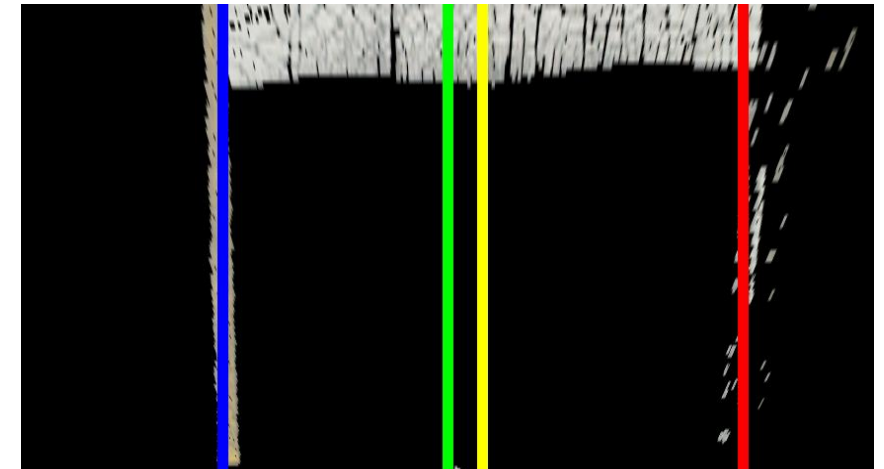
● Lane detection (Camera, OpenCV)



● 개선사항 및 개선목표



Input Camera



Lane Detection

3. 상황 인식 - Vision

● Parking ,Delivery Mission

1단계: 다중 센서 기반 환경 인식 시스템 구축	<Sensor Fusion 구성> - LiDAR (12m, 360°) : 차량 전방 하단에 최적 배치 - 다중 카메라 : 시각 기반 객체 인식
2단계: 계층적 경로 계획 시스템 개발	<환경 정보 처리> - DBSCAN 클러스터링 : 센서 노이즈 제거 - SLAM 기술 : 실시간 3D 환경맵 생성 <경로 생성> - Phase 1 : Local Path 생성 A* 알고리즘 사용 → 미션 구역에서 경로 계획 - Phase 2 : 세부 주차 경로 생성 Hybrid A* + Reeds-Shepp Curve 조합 → 좁은 공간 내 정밀 주차 경로 생성
3단계: 유형별 제어 알고리즘 구현	<주차 및 배달 미션 알고리즘> - LiDAR 포인트클라우드 기반 결정 삼각형 계산 - 카메라 원근 변환으로 주차선 인식 - MPC(Model Predictive Control)



주차 구역



Delivery Mission

[REFERENCE : 대학생 창작 모빌리티 경진대회
무인모빌리티부문 규정집]

4. Work Flow Chart

